

Patrick Semler

patricksemler@tamu.edu | [linkedin.com/in/patricksemler](https://www.linkedin.com/in/patricksemler) | github.com/patricksemler | patricksemler.dev

EDUCATION

Texas A&M University

Bachelor of Science in Computer Science

College Station, TX

Expected May 2028

- **GPA:** 3.79/4.00 | **Honors:** National Merit Scholar; President's Endowed Scholar
- **Relevant Coursework:** Data Structures and Algorithms, Program Design and Concepts, Discrete Math
- **Organizations:** Aggie Coding Club; Aggies Create; TAMUhack; TIDAL

EXPERIENCE

Machine Learning Research Assistant

Texas A&M University

June 2026 – Present

College Station, TX

- Reduced random-forest peak memory 74% (2.6 GB to 689 MB) and model size 4× while holding accuracy at 90.83% versus the original 91.06%, by tuning tree count and leaf constraints
- Expanded diagnostic coverage to swine and equine—64 new condition mappings that brought the inference pipeline to 8 animal categories—with automated validation for supported and invalid inputs
- Replaced disease-only prompting with case-conditioned generation by grounding Gemini 2.5 Flash care plans in 4 categories of patient context: symptoms, vitals, animal category, and ranked differential diagnoses

Full-Stack Developer

ApplyFinch

Jan. 2026 – Present

College Station, TX

- Shipped full stack workflows for scraping, ranking, and document generation on a multi-developer platform, collaborating through Git and SQL; React and Cloudflare Worker clients over Flask/PostgreSQL services
- Made every recommendation auditable through an end-to-end priority-ranking pipeline across Flask and React: eligibility gates plus fit, recency, and network scoring, each with a human-readable receipt
- Hardened the asynchronous pipeline against 4 failure modes—lost state, duplicate applications, transient database failures, and orphaned scraper runs—via persisted snapshots, deduplication, retries, and cross-worker cancellation
- Refactored dashboard and profile flows into typed React components, hooks, and state modules, adding Vitest, Testing Library, and Playwright coverage across authentication, onboarding, and document generation

Quantitative Developer

Maroon Fund

Feb. 2026 – May 2026

College Station, TX

- Extended risk-adjusted strategy evaluation with 3 benchmark metrics—annualized Jensen's alpha, OLS beta, and tracking error—using SPY data from a 3-source fallback pipeline
- Increased backtest fidelity by modeling slippage, commissions, and forced exits; validated the engine, metrics, and reports with an automated pytest suite

PROJECTS

LeetMind | TypeScript, Python, C++, React, FastAPI, PostgreSQL, Docker

- Built an adaptive coding-practice platform that generates and selects problems at the edge of each learner's current ability across 20 data-structure and algorithm categories
- Verified every generated problem through differential testing between the reference solution and an independent oracle across authored cases and 50 seeded randomized inputs, publishing only candidates that pass every gate
- Sandboxed Python and C++ submissions in throwaway, network-disabled Docker containers, separating user execution from expected-output evaluation so submitted code cannot inspect hidden answers
- Built 230+ automated tests across pytest, Vitest, Testing Library, and Playwright, covering learner-model math, worker concurrency, Docker judging, API contracts, and SSE

Phobos | TypeScript, Node.js, Fastify, React, PostgreSQL, Docker

- Orchestrated 8 specialized agent roles—coordination, research, planning, execution, verification, repair, module generation, and independent review—compiling durable work into Zod-validated task graphs
- Isolated generated modules in networkless, read-only Docker containers, exposing host services only through a grant-checked JSON-RPC broker so generated code cannot reach ambient credentials or arbitrary files

TECHNICAL SKILLS

Languages: Python, TypeScript, C++, SQL, HTML/CSS

Frameworks/Libraries: React, Node.js, FastAPI, Flask, Django, Fastify, scikit-learn, pandas, NumPy

Data/Infrastructure: PostgreSQL, SQLite, Supabase, Docker, Cloudflare Workers, Vercel

Tools/Protocols: Git, GitHub, pytest, Vitest, Playwright, Testing Library, OpenAPI, SSE

AI/ML: LLM Application Development, RAG (Retrieval-Augmented Generation), Gemini 2.5 Flash